

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет по лабораторной работе №4
по дисциплине
«Функциональное программирование»
Вариант 45

Студент,
группы 5130201/30102

_____ Санько В. В.

Преподаватель,
к. т. н., доц.

_____ Моторин Д. Е.

« _____ » _____ 2025г.

Санкт-Петербург, 2025

Содержание

Введение	3
Постановка задачи	4
1 Реализация	5
1.1 Определение типов и монады <code>Writer</code>	5
1.2 Логика вычислений	5
1.3 Работа с файлами без <code>do</code> -нотации	6
1.4 Главная функция	7
2 Тестирование	8
Список литературы	10

Введение

Для решения задач обработки потоковых данных в языке Haskell удобно использовать монады. Монада `Writer` позволяет выполнять вычисления, накапливая дополнительный вывод (лог), не прерывая основной поток выполнения. Это полезно при обработке массива данных, когда одна ошибочная запись не должна останавливать работу всей программы, но информацию об ошибке необходимо сохранить.

В работе реализована программа для чтения и выполнения арифметических операций из файла с использованием монады `Writer` для логирования успешных и неудачных операций. Программа реализована без использования `do`-нотации и трансформеров монад, с явным управлением файловым дескриптором.

Постановка задачи

Цель работы: Разработать на языке Haskell программу для выполнения арифметических действий, считываемых из текстового файла, с использованием монады `Writer`.

Задачи:

- Определить типы данных для представления результата вычислений и лога ошибок.
- Использовать стандартную монаду `Writer` из библиотеки `mtl` для накопления сообщений об ошибках (неверный формат строки, деление на ноль, нечисловые аргументы).
- Реализовать функции арифметических операций ($+$, $-$, $*$, $/$) над целыми числами.
- Реализовать чтение файла используя явное управление файловым дескриптором (`Handle`, `openFile`, `hClose`).
- Отказаться от использования `do`-нотации, применяя явное связывание монад ($\gg$).
- Реализовать проект с использованием сборщика `stack`.

1 Реализация

1.1 Определение типов и монады Writer

Для представления арифметических операций и выражений определены следующие типы:

```
1 data Operation = Add | Sub | Mul | Div deriving (Eq, Show)
2 data Expr = Expr Int Operation Int
```

Тип Calculation определён как монада Writer с логом в виде списка строк:

```
1 type Calculation = Writer [String] (Maybe Int)
```

Здесь Maybe Int — результат вычисления (Just значение или Nothing при ошибке), а [String] — накапливаемый лог.

1.2 Логика вычислений

Парсинг строки

Функция parseExpr преобразует строку в значение типа Maybe Expr:

```
1 parseExpr :: String -> Maybe Expr
2 parseExpr str = case words str of
3   [xStr, "+", yStr] -> buildExpr xStr Add yStr
4   [xStr, "-", yStr] -> buildExpr xStr Sub yStr
5   [xStr, "*", yStr] -> buildExpr xStr Mul yStr
6   [xStr, "/", yStr] -> buildExpr xStr Div yStr
7   _ -> Nothing
8 where
9   buildExpr xStr op yStr =
10    readMaybe xStr >>= \x ->
11    readMaybe yStr >>= \y ->
12    return (Expr x op y)
13
14 readMaybe s = case reads s of
15   [(n, "")] -> Just n
16   _ -> Nothing
```

Функция использует words для разбиения строки на части, проверяет соответствие шаблону «число оператор число» и пытается преобразовать строки в числа с помощью вспомогательной функции readMaybe. Если формат строки не соответствует ожидаемому или числа некорректны, возвращается Nothing

Вычисление выражения

Функция calculateExpr выполняет вычисление и формирует лог:

```
1 calculateExpr :: Expr -> Calculation
2 calculateExpr (Expr x op y) =
3   let result = case op of
4   Add -> Just (x + y)
```

```

5 Sub -> Just (x - y)
6 Mul -> Just (x * y)
7 Div -> if y == 0
8 then Nothing
9 else Just (x `div` y)
10 logMsg = case result of
11 Just res -> ["Success:␣" ++ show x ++ "␣" ++ opSymbol op ++ "␣" ++ show y ++
12             "␣=" ++ show res]
13 Nothing -> ["Error:␣division␣by␣zero␣in␣" ++ show x ++ "␣/" ++ show y]
14 in writer (result, logMsg)
15 where
16 opSymbol Add = "+"
17 opSymbol Sub = "-"
18 opSymbol Mul = "*"
19 opSymbol Div = "/"

```

Функция выполняет соответствующую операцию над числами. В случае деления на ноль результат становится `Nothing`, а в лог добавляется сообщение об ошибке. Для успешных операций в лог записывается информация о выполненном вычислении.

Обработка строки

Функция `processLine` объединяет парсинг и вычисление:

```

1 processLine :: String -> Calculation
2 processLine line =
3   case parseExpr line of
4   Just expr -> calculateExpr expr
5   Nothing -> writer (Nothing, ["Invalid␣line:␣" ++ line])

```

Если строка успешно парсится в выражение, выполняется вычисление. В противном случае возвращается `Nothing` с сообщением о невалидной строке в лог.

1.3 Работа с файлами без do-нотации

Для чтения файла используется явное управление дескриптором:

```

1 processFile :: FilePath -> IO [String]
2 processFile path =
3   openFile path ReadMode >>= \handle ->
4   readAllLines handle [] >>= \lines' ->
5   hClose handle >>
6   let calculations = map processLine lines'
7   resultsWithLogs = map runWriter calculations
8   allLogs = concatMap snd resultsWithLogs
9   formattedLogs = zipWith formatLine [1..] (zip lines' resultsWithLogs)
10  in return (allLogs ++ formattedLogs)
11  where
12  readAllLines :: Handle -> [String] -> IO [String]
13  readAllLines handle acc =
14    hIsEOF handle >>= \eof ->
15    if eof
16    then return (reverse acc)

```

```

17 else hGetLine handle >>= \line ->
18 readAllLines handle (line : acc)
19
20 formatLine n (line, (result, _)) =
21 "Line_" ++ show n ++ ":'" ++ line ++ "'->" ++
22 case result of
23 Just r -> "Result:" ++ show r
24 Nothing -> "Invalid"

```

Функция `processFile` открывает файл, рекурсивно читает все строки, закрывает файл, затем обрабатывает каждую строку с помощью `processLine`. Результаты и логи собираются, форматируются и возвращаются в виде списка строк. Вспомогательная функция `readAllLines` реализует рекурсивное чтение файла до достижения конца.

1.4 Главная функция

```

1 main :: IO ()
2 main =
3   putStrLn "Mathematical Operations Calculator" >>
4   putStrLn "" >>
5   processFile "operations.txt" >>= \logs ->
6   putStrLn "Execution Log:" >>
7   putStrLn (formatResult logs) >>
8   putStrLn "" >>
9   putStrLn "Statistics:" >>
10  putStrLn ("Successful operations:" ++ show (countPrefix "Success:" logs))
11    >>
12  putStrLn ("Invalid lines:" ++ show (countPrefix "Invalid line:" logs)) >>
13  putStrLn ("Errors:" ++ show (countPrefix "Error:" logs))
14  where
15    countPrefix prefix lst = length (filter (isPrefix prefix) lst)

```

Функция `main` выводит заголовок, обрабатывает файл `operations.txt`, выводит все логи и статистику по выполненным операциям. Вспомогательная функция `countPrefix` подсчитывает количество строк в логах, начинающихся с определённого префикса, что позволяет получить статистику по типам операций.

2 Тестирование

Для проверки работоспособности программы подготовлен файл `operations.txt`:

```
10 + 5
10 - 5
10 * 5
10 / 5
10 / 0
10 ++ 5
10 + five
```

Результат выполнения программы:

Mathematical Operations Calculator

Execution Log:

Success: 10 + 5 = 15

Success: 10 - 5 = 5

Success: 10 * 5 = 50

Success: 10 / 5 = 2

Error: division by zero in 10 / 0

Invalid line: 10 ++ 5

Invalid line: 10 + five

Statistics:

Successful operations: 4

Invalid lines: 2

Errors: 1

Программа корректно обрабатывает валидные арифметические операции, фиксирует ошибку деления на ноль и отмечает невалидные строки (некорректный оператор и нечисловые аргументы).

Заключение

В ходе работы была разработана программа на языке Haskell, выполняющая арифметические операции, считываемые из файла, с использованием монады `Writer` для логирования. Были выполнены следующие задачи:

- Определены типы данных для представления выражений и результатов.
- Реализована монада `Writer` для накопления логов успешных и ошибочных операций.
- Организован парсинг строк и вычисление выражений с обработкой ошибок.
- Реализовано чтение файла с явным управлением дескриптором без использования `do`-нотации.
- Программа собрана с помощью `stack` и протестирована на различных входных данных.

Программа демонстрирует эффективное использование монады `Writer` для разделения логики вычислений и механизма логирования, что повышает чистоту кода и облегчает отладку.

Список литературы

- [1] Haskell Official Documentation – официальная документация по языку Haskell [Электронный ресурс]. <https://www.haskell.org/documentation/>
- [2] Миран Липовача. Изучай Haskell во имя добра! / Пер. с англ. Леушина Д., Сеницына А., Арсанукаева Я. — М.: ДМК Пресс, 2012. — 490 с.
- [3] Официальная документация Haskell: <https://www.haskell.org> [Дата обращения: 17.11.2025]